Introduction to Profiling

Intel VTune: concepts and practice

Dr Frederick Fung

National Computational Infrastructure, Australia

2026

Acknowledgement of Country

The National Computational Infrastructure acknowledges, celebrates and pays our respects to the Ngunnawal and Ngambri people of the Canberra region and to all First Nations Australians on whose traditional lands we meet and work, and whose cultures are among the oldest continuing cultures in human history.

Workshop Goal

Overview

Measure a baseline, locate expensive code, interpret profiling evidence, and compare MPI/OpenMP execution configurations using Intel VTune and LULESH

- First hour is concepts then exercises for the remaining time.
- ARE session on Gadi.
- Open `vtune/vtune_lab.ipynb`; walk through and pause at each exercise.

This work represents the view of the author delivering this workshop.

Why bother when AI can do it for me?

Performance depends on the run

The workload, build, hardware and core placement all affect performance.

- AI can inspect code, suggest changes and help analyse profiles.
- Profiling locates measured cost in your actual workload.
- Understanding metrics helps you judge explanations and compare fairly.

Supercomputer is often a shared system, and the users are responsible for the code they put on the system running.

Timers and profilers

Timer

How long did a defined interval take?

- Baseline and variability
- End-to-end improvement

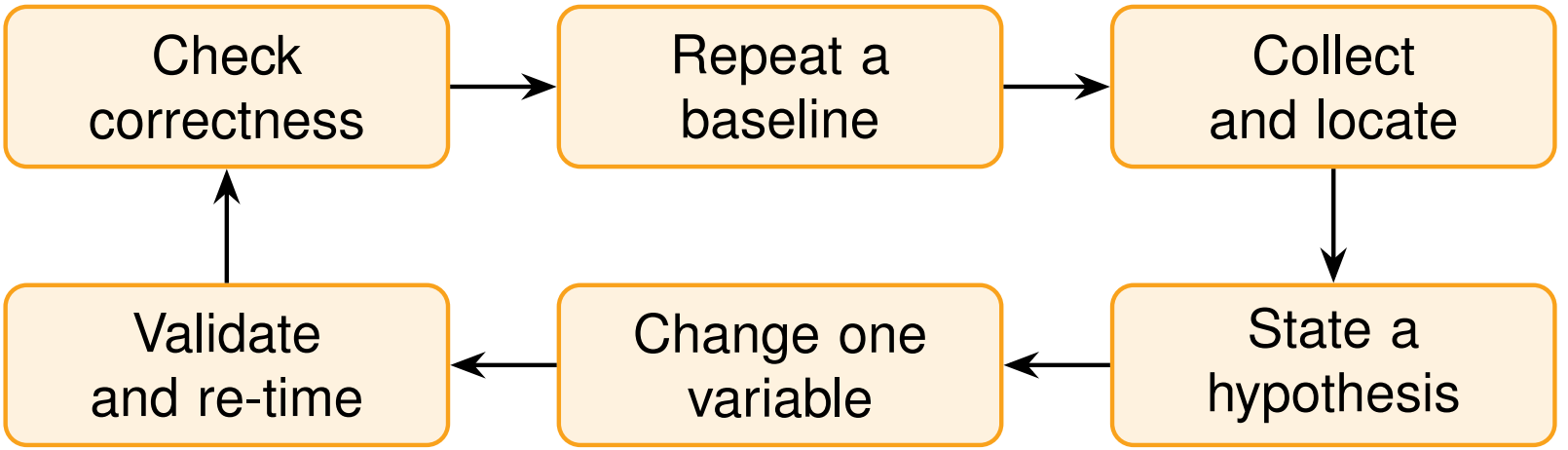
Profiler

Where did measured cost accumulate?

- Functions and callers
- Threads and hardware events

A hotspot is a candidate for investigation. Its rank in a table does not establish the cause, or the benefit of changing it.

An experiment we can repeat



Keep the comparison interpretable

Same input, executable, allocation and timing region; record the one intentional change. Repeat timings to assess variability.

Profiling mechanism: Sampling

A profiler might take a snapshot every **10 milliseconds**, recording:

- Which function is executing.
- Which source instruction is running.
- Which functions called it — the **call stack**.

Where are the samples?

Suppose the profiler collects **1,000 samples**, and **600** land inside `calculate_forces()`.

$$\text{Estimated share of CPU time} \approx \frac{600}{1000} \times 100\% = 60\%.$$

This makes `calculate_forces()` a useful place to investigate.

Sampling gives a statistical estimate of CPU time, not a count of function calls.

Software and hardware sampling

	Software sampling	Hardware sampling
Trigger	Periodic timer	A counter reaches an event threshold
Evidence	Function and call-stack costs	Cycles, instructions and other selected events
Requires	Process access	Supported CPU/VTune; counter access

Software: sample every 10 ms of thread execution.
Hardware: sample after a million occurrences of a CPU-cycle event.

Both are statistical estimates.

Profiling mechanism: Instrumentation

Instrumentation adds measurement code at selected points to:

- Record a timestamp when a function starts.
- Record another timestamp when it returns.
- Count calls and accumulate their measured durations.

What was recorded?

Suppose instrumentation records **100 sequential calls** to `calculate_forces()`, taking **2 seconds** in total.

$$\text{Average elapsed time per call} = \frac{2\text{ s}}{100} = 20\text{ ms}.$$

We know how often the function was called and its average measured duration.

Timestamp differences measure elapsed time. The measurement code itself adds overhead and can affect execution.

Build with source information

```
# From the intro-to-profiling repository:
BUILD_JOBS=4 bash build_scripts/build_lulesh_vtune.sh \
  /scratch/vp91/$USER/LULESH
```

- `-O3`: representative optimised execution.
- `-g`: symbols and source locations.
- `-fno-omit-frame-pointer`: help stack unwinding.
- Enable MPI and OpenMP; use the same MPI at run time.

LULESH excutable is pre-build and provided through module in this workshop.

LULESH workload

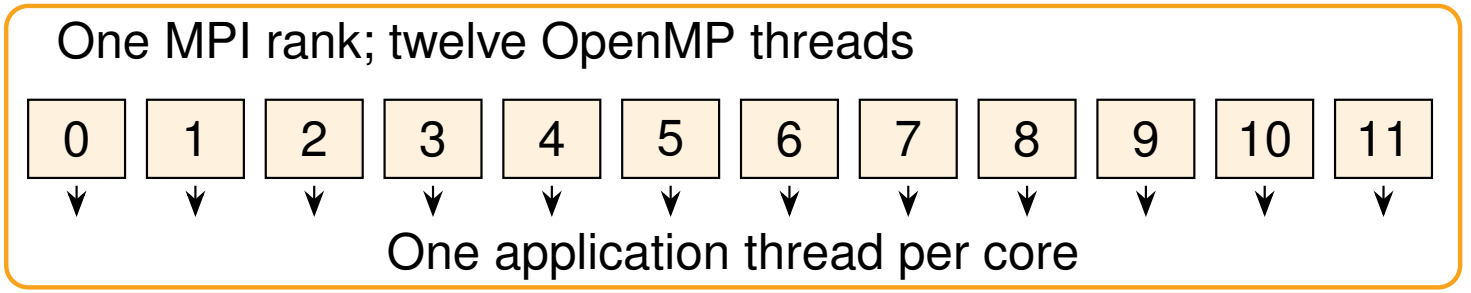
- C++ proxy for a hydrodynamics calculation.
- MPI divides the mesh; OpenMP works within a rank.
- `-s N`: side length of the *local* mesh.
- `-i K`: iteration limit; check actual iterations too.

$$\text{total mesh elements} = \text{MPI ranks} \times N^3.$$

Use a valid decomposition

Rank counts must be perfect cubes: 1, 8, 27, ... The core exercises use one MPI rank and vary OpenMP threads.

The twelve-core application run




```
export OMP_NUM_THREADS=12 OMP_DYNAMIC=FALSE
export OMP_PLACES=cores OMP_PROC_BIND=close
mpirun -np 1 --map-by slot:PE=12 --bind-to core \
  --report-bindings lulesh2.0 -s 30 -i 200
```

- `slot`: fill a node's available process slots before the next node.
- `PE=12`: twelve processing elements per rank; cores here.
- `-bind-to core`: bind the rank to that set of twelve cores.
- OpenMP settings request twelve threads and bind them to core places within the rank's set.

For *p* threads, match `OMP_NUM_THREADS=`*p* with `PE=`*p*. The ARE allocation stays at twelve cores.

Software Hotspots

```
export OMP_NUM_THREADS=1 OMP_DYNAMIC=FALSE
export OMP_PLACES=cores OMP_PROC_BIND=close
mpirun -np 1 --map-by slot:PE=1 --bind-to core \
  vtune -collect hotspots -knob sampling-mode=sw \
  -knob enable-characterization-insights=false \
  -trace-mpi -result-dir /path/to/fresh/result \
  -- lulesh2.0 -s 30 -i 200
```

- Software sampling with additional hardware insights disabled.
- The MPI launcher starts VTune for the application rank.
- `-trace-mpi` supports MPI-aware Open MPI results.
- Use a fresh result location for each run.

Hardware Hotspots

Reference only; this command is not a notebook exercise.

```
export OMP_NUM_THREADS=1 OMP_DYNAMIC=FALSE
export OMP_PLACES=cores OMP_PROC_BIND=close
mpirun -np 1 --map-by slot:PE=1 --bind-to core \
  vtune -collect hotspots -knob sampling-mode=hw \
  -knob enable-stack-collection=false \
  -trace-mpi -result-dir /path/to/fresh/hardware-result \
  -- lulesh2.0 -s 30 -i 200
```

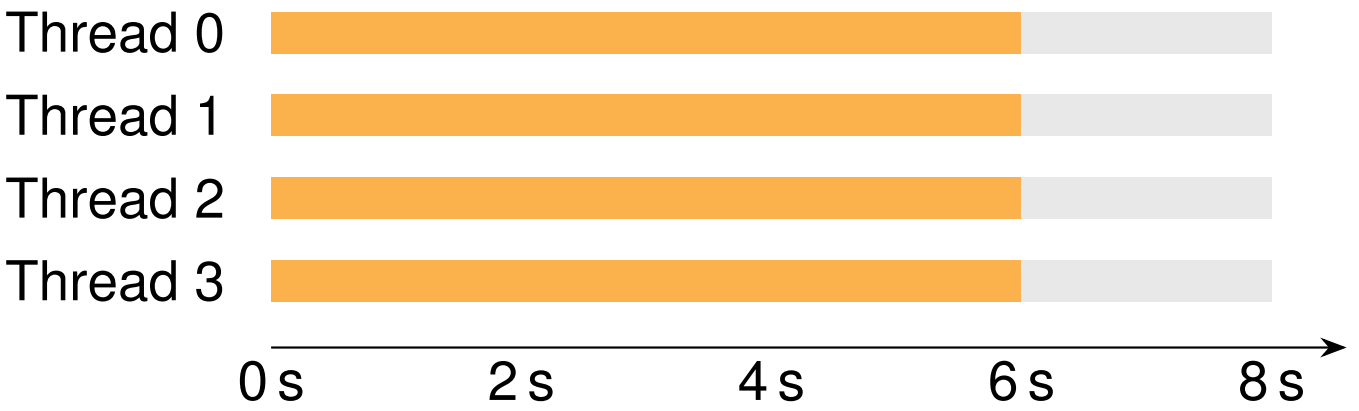
- `sampling-mode=hw` selects hardware event sampling.
- This example disables call stacks; counters are still required.
- CPU support, VTune version and collection permissions matter.

Collect, then report

```
# Use the exact result path, including a hostname suffix.
vtune -report summary -result-dir /path/to/result.host
vtune -report hotspots -group-by function \
  -result-dir /path/to/result.host
vtune-gui /path/to/result.host
```

- Summary: overall collection metrics and warnings.
- Hotspots: costs grouped by function.
- The notebook saves reports under your own scratch directory.

CPU time can exceed elapsed time



$$T_{\text{CPU}} = 4 \times 6 = 24 \text{ s}, \quad T_{\text{elapsed}} = 8 \text{ s}.$$

Average on-CPU concurrency is $24/8 = 3$ threads.

On-CPU activity can include spin. This arithmetic does not establish useful work or explain the inactive intervals.

Interpret the timing categories carefully

CPU time	Accumulated on-CPU thread time.
Effective time	Excludes recognised spin and runtime overhead; does not prove all work is productive.
Spin time	Recognised active waiting, consuming CPU time.
Overhead time	Recognised runtime management; not all profiler overhead.
Paused time	Collection paused; not general sleeping-thread time.
Thread count	Observed threads, possibly including helper threads.

Availability depends on the analysis and runtime. Missing metrics are not zero.

Always ask: percentage of what?

An illustrative result reports 1.63 effectively used physical cores out of 48 detected physical cores:

$$100 \times \frac{1.63}{48} \approx 3.4\%.$$

- The denominator may describe the entire node.
- How many cores did this job actually request?
- Physical cores and logical CPUs are different denominators.
- A utilisation percentage is not measured parallel efficiency.

For a fixed workload and one MPI rank

Using unprofiled elapsed times: $S_p = T_1/T_p$, and $E_p = S_p/p$, where p is the OpenMP thread count.

Speedup and parallel efficiency

Fixed workload; one MPI rank; twelve cores allocated.
p is the number of OpenMP threads used in the run.

- T_1: median unprofiled elapsed time with one thread.
- T_p: median unprofiled elapsed time with p threads.

$$S_p = \frac{T_1}{T_p} \qquad E_p = \frac{S_p}{p} = \frac{T_1}{pT_p}.$$

S_p measures how many times faster the run is; E_p is the fraction of ideal linear speedup. Ideally, $S_p = p$ and $E_p = 1 = 100\%$.

Illustrative example; not measured workshop timings

If $T_1 = 12\,\text{s}$ and $T_{12} = 2\,\text{s}$, then

$$S_{12} = 12/2 = 6, \qquad E_{12} = 6/12 = 0.5 = 50\%.$$

Twelve threads give a sixfold speedup: half the ideal linear speedup.

Follow cost back to the code

1. **Summary**: confirm application, interval and warnings.
2. **Bottom-up**: choose a function; group by process/thread.
3. **Caller/Callee**: understand the path that reaches it.
4. **Source/Assembly**: locate a sampled operation or loop.
5. **Timeline**: investigate phases hidden by an average.

Why might speedup disappoint?

Candidate explanation	Evidence to investigate
Too little work per thread	Larger mesh; same comparison protocol
Serial phases or imbalance	Timeline and work distribution
Runtime waiting	Spin/runtime functions and their callers
Data-access limits	Hardware memory metrics in the hot region
Launch overhead or noise	Application timer; repeated timings

A flat speedup curve alone does not distinguish these explanations. Higher CPU time can coexist with lower elapsed time.

Choose an analysis from a question

Question	Collection	Look for
Where is CPU time spent?	`hotspots`	Functions and call paths
How are compute resources used?	`hpc-performance`	CPU, memory and floating-point metrics
Is access to data limiting a hot region?	`memory-access`	Stalls, bandwidth, cache and NUMA metrics

Hardware support matters

The last two analyses need supported hardware counters and access permissions. Available metrics depend on CPU, version and runtime.

HPC Performance

```
export OMP_NUM_THREADS=4 OMP_DYNAMIC=FALSE
export OMP_PLACES=cores OMP_PROC_BIND=close
mpirun -np 1 --map-by slot:PE=4 --bind-to core \
  vtune -collect hpc-performance -trace-mpi \
  -result-dir /path/to/fresh/hpc \
  -- /path/to/lulesh2.0 -s 30 -i 200
```

- Inspect available CPU, memory and floating-point metrics.
- Detailed OpenMP metrics depend on runtime support; a GCC build may not expose Intel runtime region metrics.

Memory Access: bandwidth and latency

```
# Same four-thread environment as the HPC collection.
mpirun -np 1 --map-by slot:PE=4 --bind-to core \
  vtune -collect memory-access -trace-mpi \
  -result-dir /path/to/fresh/memory \
  -- /path/to/lulesh2.0 -s 30 -i 200
```

- Bandwidth: data transferred per second.
- Latency: time to satisfy an access.
- Low bandwidth can coexist with memory stalls.
- Investigate locality, reuse and independent accesses in the hot code.